SOFTWARE REQUIREMENTS SPECIFICATION

FOR

Ambulance Booking System

Prepared by

ADITHYA SHYAM D (241IT005)

DEV PATEL (241IT023)

RITESH BHAGAT (241IT063)

DEPARTMENT OF INFORMATION TECHNOLOGY

NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA SURATHKAL

Table of Contents

1 Introduction **1**
1.1 Purpose 1
1.2 Scope 1
1.3 Definitions, Acronyms, and Abbreviations 1
1.4 References 2
1.5 Overview 2
2 Overall Description **2**
2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Classes 3
2.4 Constraints 3
2.5 Assumptions and Dependencies 4
3 Specific Requirements **4**
3.1 Functional Requirements 4
3.2 External Interface Requirements 6
3.2.1 User Interfaces 6
3.2.2 Hardware Interfaces 6
3.2.3 Software Interfaces 6
3.2.4 Communications Interfaces 7
3.3 Other Non-Functional Requirements 7
3.4 Design Constraints 7
Appendix A **8**
Index **9**
Version **10**

1. Introduction

1.1 Purpose

The purpose of this project is to develop a cross-platform Ambulance Booking System that connects people in medical emergencies with nearby ambulance drivers and organisations in the shortest possible time. The system is designed to allow a patient or bystander to request a suitable ambulance, automatically match the request to available drivers, and notify the relevant driver or ambulance organisation instantly through a WhatsApp messaging integration. The application provides an intuitive interface for booking, live tracking, and status updates, and it separates access into three roles: user, ambulance driver or organisation, and administrator. This platform is intended to serve the general public, ambulance service providers, and hospital administrators who need a reliable dispatch and coordination tool.

1.2 Scope

The project focuses on building an end-to-end booking and dispatch workflow through a mobile application backed by a server-side application and database. Users can register or sign in, choose the type of ambulance they need, share their pickup location, and place a request that is routed to the nearest available driver. Drivers and organisations receive requests through in-app notifications and a WhatsApp alert, and can accept, reject, and update the status of a trip. The current scope includes account management, role-based access, ambulance-type selection, driver matching, WhatsApp-based notification, live status tracking, trip completion, and an administrative dashboard. Future extensions may include:

- Integration with hospital bed-availability and emergency-room systems.
- In-app payment settlement, insurance verification, and automated invoicing.
- AI-based demand prediction and dynamic positioning of idle ambulances.
- Multilingual voice-based booking for accessibility.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
API	Application Programming Interface
RBAC	Role-Based Access Control
JWT	JSON Web Token, used for stateless authentication
GPS	Global Positioning System, used for location capture
ETA	Estimated Time of Arrival
BLS	Basic Life Support ambulance
ALS	Advanced Life Support ambulance
ICU	Intensive Care Unit ambulance
OTP	One-Time Password
REST	Representational State Transfer, an API architectural style
WhatsApp Business API	Messaging interface used to send booking alerts to drivers and organisations
Dispatch	Process of assigning and routing an ambulance to a request
CRUD	Create, Read, Update, Delete database operations
UI / UX	User Interface / User Experience

1.4 References

[1] IEEE, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998.

[2] Meta, *WhatsApp Business Platform: Cloud API Documentation.* [Online]. Available: https://developers.facebook.com/docs/whatsapp. [Accessed: Aug. 24, 2025].

[3] Google, *Google Maps Platform Documentation.* [Online]. Available: https://developers.google.com/maps/ [Accessed: Aug. 24, 2025].

[4] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures.* Doctoral dissertation, University of California, Irvine, 2000.

[5] Ministry of Health and Family Welfare, Government of India, *Emergency Medical Services (108) Operational Guidelines.* [Online]. Accessed: Aug. 23, 2025.

1.5 Overview

The following sections of this document present a detailed description of the system requirements, covering both functional and non-functional aspects. They outline the product perspective, the interfaces between the application and external services such as the WhatsApp Business API and mapping providers, the performance and reliability needs of an emergency-oriented system, and the design constraints that guide development. Supporting information such as a workflow block diagram, an index of key terms, and a version history is provided in the appendices to ensure clarity and completeness of the specification.

2. Overall Description

2.1 Product Perspective

The Ambulance Booking System is a client-server software application built for emergency medical transport coordination. The frontend is delivered as a mobile application for Android and iOS, while the backend exposes RESTful services and manages a central database of users, drivers, organisations, and bookings. The product functions as a real-time dispatch platform that reduces the time between a request and the arrival of an ambulance by automating driver matching and notification.

The system is dependent on external services including the WhatsApp Business API for driver alerts, a mapping and geolocation provider for pickup detection and routing, and a network connection for real-time communication. It can be integrated into larger healthcare ecosystems such as hospital admission systems, government emergency-response services, and private ambulance fleet-management tools. The application supports role-based access for three user classes and maintains persistent records of every booking for auditing and analytics.

2.2 Product Functions

Account and Access Management

- Register new users and authenticate returning users.
- Provide separate login flows for user, driver/organisation, and administrator roles.

Booking and Dispatch

- Allow users to select an ambulance type and share pickup location.

- Match a request to the nearest available driver or organisation.
- Notify the assigned driver or organisation through a WhatsApp alert and in-app notification.

Trip Coordination

- Let drivers accept or reject requests and update trip status.
- Provide live tracking and an estimated time of arrival to the user.

Records, Reporting, and Administration

- Store booking history and trip records for each user and driver.
- Collect ratings and feedback after trip completion.
- Provide an administrative dashboard to manage drivers, organisations, and bookings, and to view operational analytics.

2.3 User Classes

- **Users / Patients:** Individuals or bystanders who request an ambulance, choose the required type, track its arrival, and provide feedback after the trip.
- **Drivers / Ambulance Organisations:** Service providers who receive dispatch requests, accept or reject them, navigate to the pickup point, and update trip status. An organisation account may manage several drivers and vehicles.
- **Administrators:** System operators who verify and manage driver and organisation accounts, monitor active bookings, resolve disputes, and review analytics and audit logs.

2.4 Constraints

- **C.1 Regulatory Policies** – Must comply with applicable data-protection and healthcare regulations, including secure handling of personal and location data under the Digital Personal Data Protection framework.
- **C.2 Connectivity Constraints** – Requires an active internet connection on both user and driver devices; degraded or offline networks reduce real-time capability.
- **C.3 Software Interfaces** – Dependent on the WhatsApp Business API, a mapping/geolocation provider, and a payment gateway for full functionality.
- **C.4 Hardware Requirements** – Requires GPS-enabled devices for accurate pickup detection and driver tracking.
- **C.5 Audit Functions** – Logs all bookings, dispatch decisions, status changes, and administrative actions for reproducibility and dispute resolution.
- **C.6 Control Functions** – Requires request timeouts, automatic reassignment on driver rejection, and validation of driver availability before dispatch.
- **C.7 Criticality of the Application** – As an emergency service, reliability and low latency are critical, since delays or failures can directly affect patient safety.

2.5 Assumptions and Dependencies

- The system assumes that drivers and organisations keep their availability status and location updated while on duty.
- It is assumed that users and drivers have GPS-enabled smartphones with an active internet connection.
- The WhatsApp notification feature depends on the availability of the WhatsApp Business API and valid, opted-in recipient numbers.
- Location detection and routing depend on a third-party mapping provider being reachable.
- If pricing rules, regulatory requirements, or third-party API terms change, the product requirements must be updated to accommodate the new conditions.
- Payment settlement depends on the availability and correct configuration of an external payment gateway.

3. Specific Requirements

3.1 Functional Requirements

F.1 User Registration and Authentication

Description: The system shall allow new users to register with basic details and phone-number or OTP verification, and shall authenticate returning users on subsequent sign-in.

Input: User details (name, phone number, password/OTP).

Output: Confirmation of successful registration or authenticated session.

F.2 Role-Based Login

Description: The system shall present a login screen that asks whether the person is a new or existing user, and shall route authenticated users into one of three modes: user, driver/organisation, or administrator, granting access based on role.

Input: Login credentials and selected role.

Output: Role-specific dashboard and access scope.

F.3 Select Ambulance Type and Place Booking

Description: The system shall allow a user to select the required ambulance type (for example BLS, ALS, ICU, or patient transport) and place a booking request.

Input: Selected ambulance type and booking request.

Output: A created booking with a unique request identifier.

F.4 Capture Pickup Location

Description: The system shall capture the user's pickup location using GPS, and allow manual entry or map-pin adjustment when GPS is unavailable.

Input: GPS coordinates or manually entered address.

Output: Validated pickup location attached to the booking.

F.5 Match Nearest Available Ambulance

Description: The system shall identify and rank available drivers or organisations near the pickup location based on distance, availability, and ambulance type, and assign the most suitable one.

Input: Pickup location, requested type, and live driver availability.

Output: An assigned driver or organisation for the request.

F.6 Notify Driver via WhatsApp API

Description: The system shall notify the assigned driver or ambulance organisation of a new request through a WhatsApp Business API message, in parallel with an in-app notification, including pickup details and request type.

Input: Assigned driver contact and booking details.

Output: A delivered WhatsApp alert and in-app notification to the driver/organisation.

F.7 Accept or Reject a Request

Description: The system shall allow the driver or organisation to accept or reject an incoming request within a time limit, and shall automatically reassign the request to the next-best candidate on rejection or timeout.

Input: Driver's accept/reject response.

Output: Updated booking assignment and status.

F.8 Live Tracking and ETA

Description: The system shall provide the user with the live location of the assigned ambulance and a continuously updated estimated time of arrival once a request is accepted.

Input: Driver's live GPS updates and route data.

Output: Real-time map position and ETA displayed to the user.

F.9 Trip Status Updates

Description: The system shall let the driver update trip status through defined stages (assigned, en route, arrived, patient on board, completed), and shall reflect each change to the user in real time.

Input: Driver-triggered status changes.

Output: Synchronised trip status for user, driver, and administrator.

F.10 Payment and Billing

Description: The system shall compute the fare based on distance, ambulance type, and applicable rules, and shall support payment through an integrated gateway or cash-on-completion.

Input: Completed trip details and selected payment method.

Output: A generated invoice and recorded payment status.

F.11 Booking History and Records

Description: The system shall maintain a history of past bookings for each user and driver, accessible from their respective dashboards.

Input: User or driver identity.

Output: A list of past bookings with details and status.

F.12 Rating and Feedback

Description: The system shall allow users to rate and provide feedback on a completed trip, and shall aggregate ratings for each driver or organisation.

Input: User rating and optional comments.

Output: Stored feedback and updated driver rating.

F.13 Administrative Management

Description: The system shall provide administrators with tools to verify and manage driver and organisation accounts, monitor active and past bookings, and view operational analytics such as response times and demand patterns.

Input: Administrator actions and query filters.

Output: Updated account states, dispatch overview, and analytics reports.

F.14 Notifications and Alerts

Description: The system shall send notifications for key events, including request confirmation, driver assignment, arrival, and completion, through in-app messages and WhatsApp where applicable.

Input: Booking lifecycle events.

Output: Timely notifications delivered to the relevant parties.

3.2 External Interface Requirements

3.2.1 User Interfaces

Description: A graphical mobile-app interface shall be provided for registration, role-based login, ambulance-type selection, booking, live tracking, and history. The interface shall present a map view for pickup selection and ambulance tracking, and role-specific dashboards for users, drivers/organisations, and administrators.

Input: User interactions such as login, ambulance-type selection, and location confirmation.

Output:

- Confirmation of booking and driver assignment.
- Live map with ambulance position and ETA.
- Trip status updates, invoices, and booking history.

3.2.2 Hardware Interfaces

Description: The system shall run on GPS-enabled smartphones and standard computers with network access. Driver and user devices shall use their location sensors for pickup detection and live tracking; no specialised hardware beyond a commodity smartphone or computer is required.

3.2.3 Software Interfaces

Description: The system shall interface with external services and the local application stack for its core functions.

Input:

- WhatsApp Business API for driver and organisation alerts.
- Mapping and geolocation provider for location, routing, and ETA.
- Payment gateway for fare settlement.
- Backend framework and database for persistence (for example, a REST backend with a relational database).

Output:

- Dispatch messages, booking records, and trip logs.

3.2.4 Communications Interfaces

Description: The system shall provide communication between the frontend and backend through RESTful APIs over HTTPS. All requests and responses shall use JSON format, and real-time updates such as tracking and status changes shall use a persistent channel (for example WebSockets or push notifications). Driver alerts shall be delivered through the WhatsApp Business API. Authentication shall be enforced using tokens (for example JWT) for secure access.

Input: API requests including registration, booking, and status updates.

Output: API responses containing booking confirmations, driver location, ETA, and trip status.

3.3 Other Non-Functional Requirements

NF.1 Availability and Reliability
Description: As an emergency service, the system shall target high availability (for example 24/7 operation with minimal downtime) and shall handle driver-rejection or timeout by automatic reassignment so that a request is not left unserved.

NF.2 Performance
Description: The system shall dispatch a request and deliver the driver notification within a few seconds under normal load, and shall update live tracking and ETA with low latency to support time-critical decisions.

NF.3 Security and Privacy
Description: The system shall enforce authenticated, role-based access, encrypt data in transit over HTTPS, and protect personal and location data in line with applicable data-protection regulations. API access shall be secured using tokens and keys.

NF.4 Scalability
Description: The system shall scale to support growing numbers of concurrent users, drivers, and bookings across multiple regions without degrading dispatch performance.

NF.5 Platform Support
Description: The system shall be available as a mobile application on Android and iOS devices, so users, drivers, and administrators can install and operate it across a wide range of common smartphones.

NF.6 Usability
Description: The booking flow shall be simple enough to complete under stress in a few steps, with clear prompts and prominent action controls, since users may be in distress during an emergency.

3.4 Design Constraints

Third-Party Dependency: Core notification, mapping, and payment functions depend on external providers (WhatsApp Business API, mapping provider, payment gateway); their availability and terms of use directly affect the system.

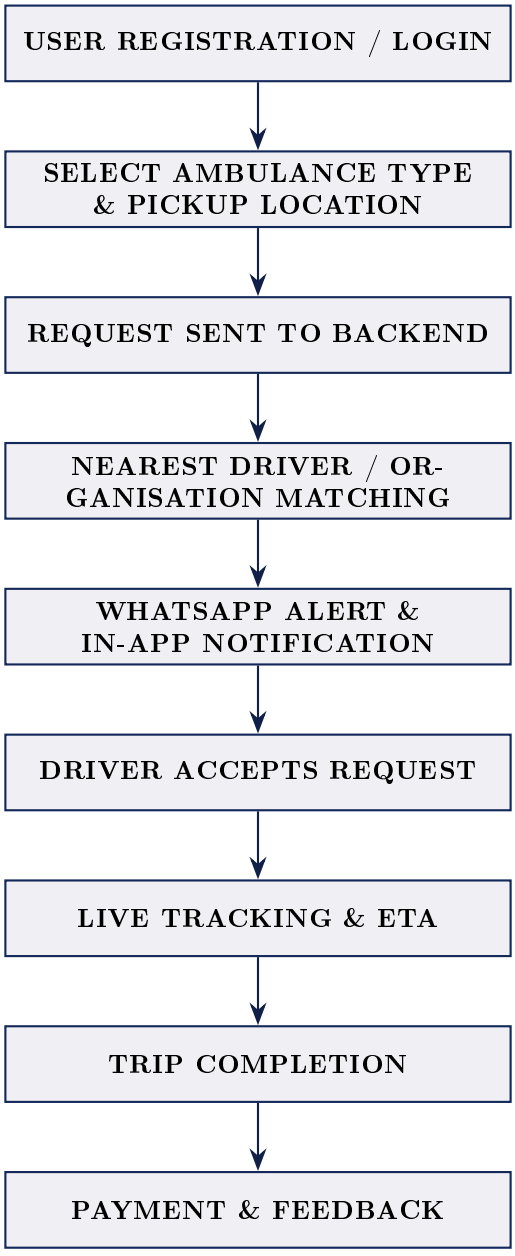
Connectivity Requirement: The application requires an active internet connection for real-time dispatch and tracking; offline operation is out of scope for the current design.

Regulatory Requirement: Handling of personal and health-related data must follow applicable privacy and healthcare regulations, and this compliance is a fixed constraint on the design.

Appendix A

A.1 Block Diagram

The block diagram below illustrates the overall workflow of the proposed system. The process begins with the user registering or logging in and selecting the required ambulance type along with a pickup location. The request is sent to the backend, which matches it to the nearest available driver or organisation. A WhatsApp alert and in-app notification are sent to the assigned driver, who accepts the request. The user then receives live tracking and an estimated time of arrival until the trip is completed, after which payment and feedback are recorded.



Index

A
API – Application Programming Interface
Admin Mode – Role for managing drivers, organisations, and bookings
Ambulance Type – Category of vehicle requested (BLS, ALS, ICU, transport)

B
Booking – A user request for an ambulance with a unique identifier
BLS / ALS – Basic and Advanced Life Support ambulances

D
Dispatch – Assigning and routing an ambulance to a request
Driver Mode – Role for accepting requests and updating trip status

E
ETA – Estimated Time of Arrival

G
GPS – Global Positioning System, used for location and tracking

J
JWT – JSON Web Token, used for authentication

L
Live Tracking – Real-time display of ambulance location

R
RBAC – Role-Based Access Control
REST – Representational State Transfer API style

U
User Mode – Role for requesting and tracking an ambulance

W
WhatsApp Business API – Messaging interface for driver alerts

Version

Version	Last updated	Reason for change
1.0	03-08-2026	Initial draft

DEPARTMENT OF INFORMATION TECHNOLOGY

NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA, SURATHKAL

Data Flow Diagrams

for

Ambulance Booking System

(derived from the Software Requirements Specification)

Prepared by: Adithya Shyam D (241IT005)
Dev Patel (241IT023)
Ritesh Bhagat (241IT063)

Team 16

August 2026

Contents

1 Introduction **1**

1.1 Notation 1

1.2 Data Stores 1

1.3 Requirement-to-Process Traceability 1

2 Level 0: Context Diagram **2**

3 Level 1: Top-Level DFD **3**

4 Level 2: Process 0.1 — Account and Access **4**

5 Level 2: Process 0.2 — Booking and Location **4**

6 Level 2: Process 0.3 — Matching and Dispatch **5**

7 Level 2: Process 0.4 — Trip Tracking and Status **5**

8 Level 2: Process 0.5 — Billing, History and Feedback **6**

9 Level 2: Process 0.6 — Administration and Analytics **6**

1 Introduction

This document presents the Data Flow Diagram (DFD) model for the Ambulance Booking System. The model is derived directly from the functional requirements of the Software Requirements Specification (SRS) and expresses how data moves between external actors, processes, and data stores.

The model is levelled. The context diagram (Level 0) treats the whole system as a single process. It is decomposed into six major processes at Level 1, and *each* of those six processes is expanded into its own Level 2 diagram, giving six Level 2 diagrams in total (Sections 5–10). The external service entities of the context diagram (Maps / GPS Service, Messaging Service, Payment Gateway) reappear only in the Level 2 diagrams where they actually take part, and no new external entity is introduced below Level 0.

1.1 Notation

- **External entity** — solid rounded rectangle (light orange). A person, organisation, or third-party service outside the system boundary.
- **Process** — circle (light blue), numbered according to its level.
- **Data store** — open-ended box with two horizontal rules (light green), numbered D1–D5.
- **Inter-process connector** — solid rounded rectangle (light purple). This is *not* an external entity; it marks a flow that continues into, or arrives from, another Level 1 process. It lets each Level 2 diagram be read on its own without redrawing its neighbours.
- **Data flow** — labelled arrow. A two-way exchange is drawn as two separate parallel arrows, and every label is placed on the outer side of its own arrow, so each label belongs to exactly one arrow.

Where an external entity both starts and ends a chain of processes it is drawn twice in the same diagram (for example the actor in Section 5). This is standard SA/SD practice and is used here so that no flow has to cross another.

Data stores are introduced at Level 2, where the processes that read and write them are visible; the Level 1 diagram shows only the process-to-process and process-to-entity flows, which keeps it readable.

1.2 Data Stores

Store	Name	Contents
D1	User Account & Role Data	Registered users, drivers, organisations and administrators; credentials, roles, verification status
D2	Availability & Vehicle Data	Ambulance types, vehicle registration, live availability and last known location of each provider
D3	Booking & Assignment	Booking requests, pickup and drop details, assigned provider, booking state
D4	Trip Status & Location	Live GPS trace, trip milestones and ETA history for each active trip
D5	Payments, Invoices & Ratings	Fare breakdowns, invoices, payment results, ratings and feedback

Table 1: Data stores used in the model

1.3 Requirement-to-Process Traceability

The table below maps each functional requirement in the SRS to the DFD process that realises it, so the diagrams can be checked against the specification.

SRS Functional Requirement	DFD Process
F.1 User Registration and Authentication	0.1.1 Register & Verify OTP
F.2 Role-Based Login	0.1.2 Authenticate Credentials; 0.1.3 Create Session & Assign Role
F.3 Select Ambulance Type and Place Booking	0.2.1 Capture Type & Pickup; 0.2.3 Create Booking Record
F.4 Capture Pickup Location	0.2.2 Resolve & Validate Location
F.5 Match Nearest Available Ambulance	0.3.1 Find & Rank Providers
F.6 Notify Driver via WhatsApp API	0.3.2 Notify Provider & Await Reply
F.7 Accept or Reject a Request	0.3.2 Notify Provider & Await Reply; 0.3.3 Confirm Assignment
F.8 Live Tracking and ETA	0.4.2 Calculate ETA & Route
F.9 Trip Status Updates	0.4.1 Receive GPS & Status; 0.4.3 Publish Status & Alerts
F.10 Payment and Billing	0.5.1 Calculate Fare; 0.5.2 Process Payment
F.11 Booking History and Records	0.5.3 Record History & Feedback
F.12 Rating and Feedback	0.5.3 Record History & Feedback
F.13 Administrative Management	0.6.1 Verify & Manage Providers; 0.6.2 Monitor Bookings
F.14 Notifications and Alerts	0.3.2 Notify Provider; 0.4.3 Publish Status & Alerts; 0.6.3 Reports & Analytics

Table 2: Traceability of SRS functional requirements to DFD processes

2 Level 0: Context Diagram

The context diagram treats the whole Ambulance Booking System as a single process and shows its interaction with the external users (User / Patient, Driver / Organisation, Administrator) and the external services (Maps / GPS, Messaging, Payment Gateway). Each two-way exchange is drawn as two separate arrows that leave the entity and the system at different points, so that no arrow and no label sits on another.

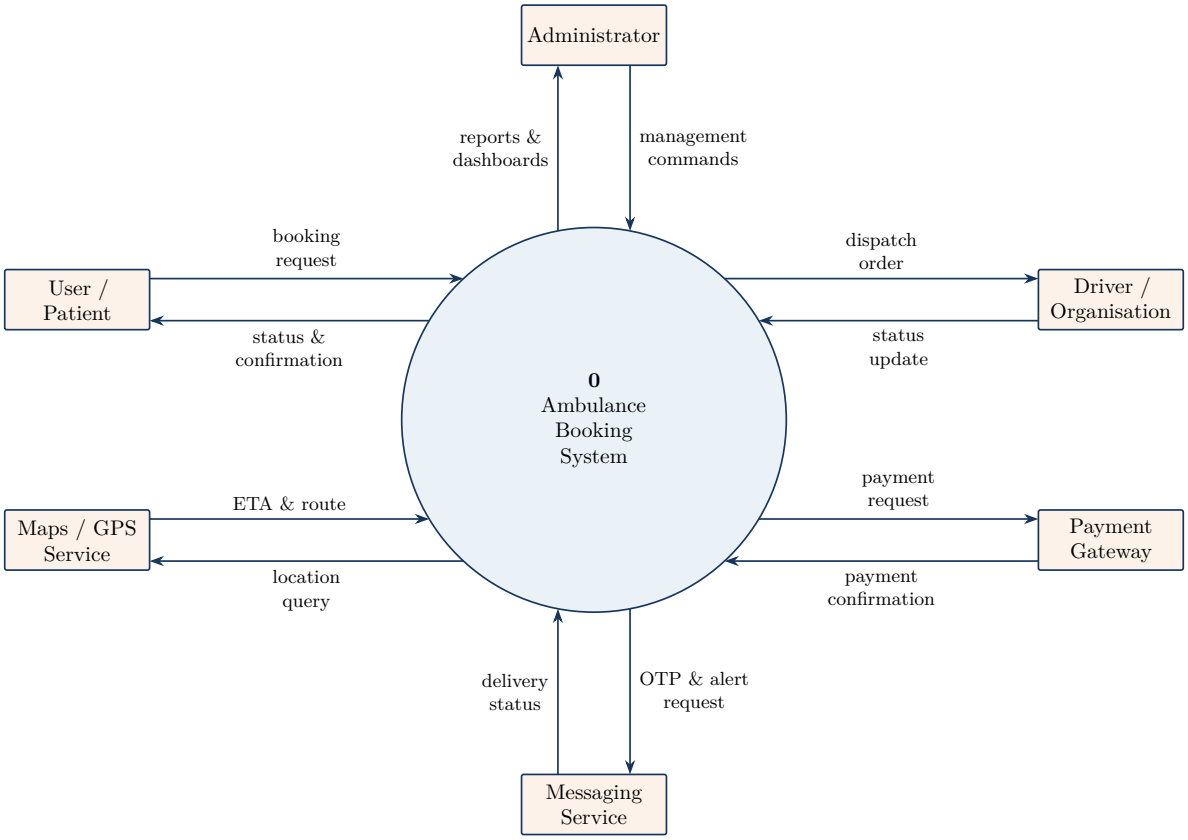


Figure 1: Level 0 DFD: Context diagram. The Messaging Service covers WhatsApp, push and SMS/OTP delivery.

3 Level 1: Top-Level DFD

The Level 1 DFD separates the system into six major processes that together cover the SRS functional requirements: account and access (F.1, F.2), booking and location (F.3, F.4), matching and dispatch (F.5–F.7), trip tracking and status (F.8, F.9), billing, history and feedback (F.10–F.12), and administration and analytics (F.13, F.14). Each of these six processes is expanded in its own Level 2 diagram in the following six sections.

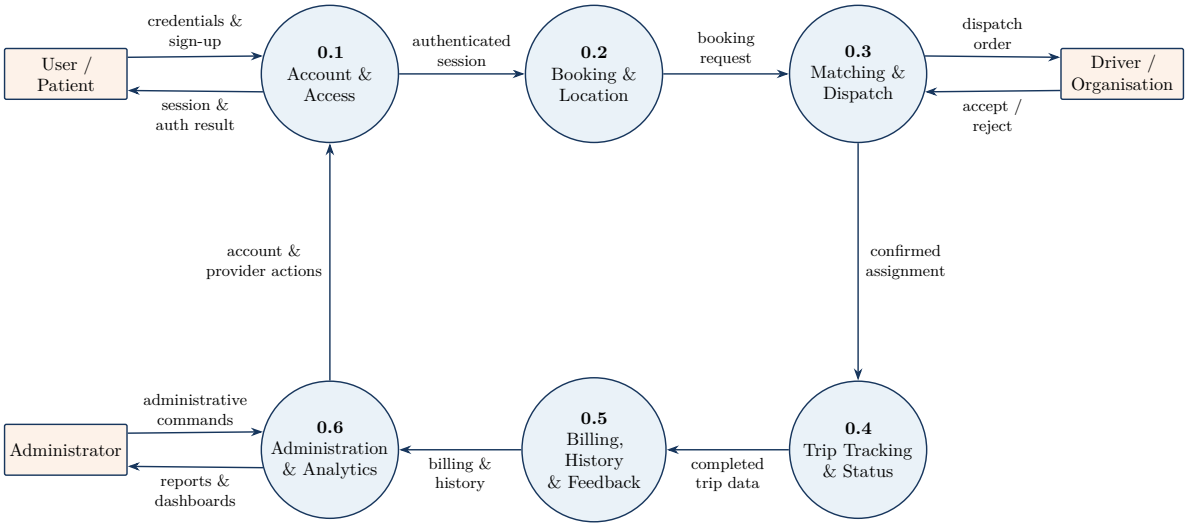


Figure 2: Level 1 DFD: the six major processes. Data stores appear in the Level 2 diagrams.

4 Level 2: Process 0.1 — Account and Access

This diagram expands Process 0.1 and realises **F.1** (registration and authentication) and **F.2** (role-based login). A new user registers and is verified by a one-time password delivered through the Messaging Service. The credentials are then checked against store D1, and a session carrying the user's role is created and returned. The actor entity is drawn twice — once as the source of the sign-up and login data, once as the recipient of the session — so that no flow has to cross another.

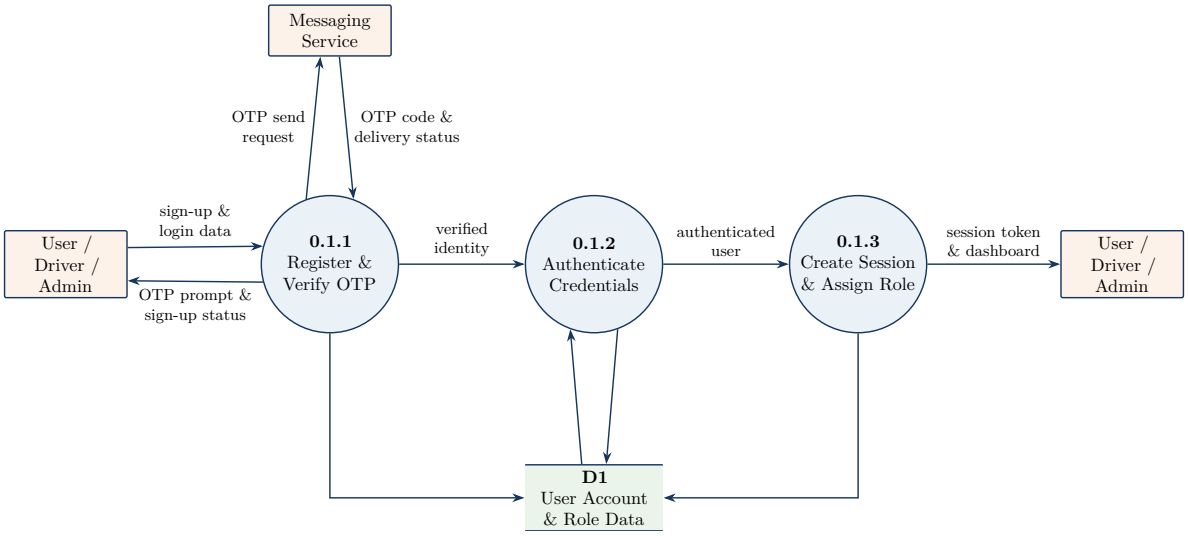


Figure 3: Level 2 DFD for Process 0.1: Account and access (F.1, F.2)

5 Level 2: Process 0.2 — Booking and Location

This diagram expands Process 0.2 and realises **F.3** (select ambulance type and place a booking) and **F.4** (capture the pickup location). The pickup point supplied by the user is geocoded and validated against the Maps / GPS Service before a booking record is written to D3 and passed on to Process 0.3.

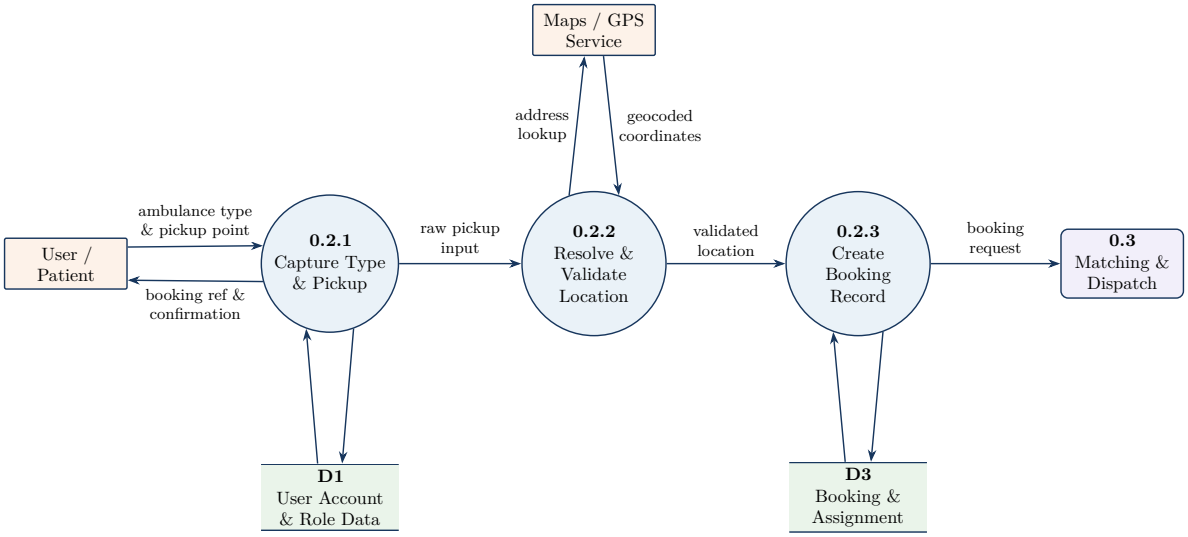


Figure 4: Level 2 DFD for Process 0.2: Booking and location capture (F.3, F.4)

6 Level 2: Process 0.3 — Matching and Dispatch

This diagram expands Process 0.3 and realises **F.5**–**F.7**: ranking the nearest available providers from D2 (F.5), notifying the selected provider through the Messaging Service (F.6), and handling the accept, reject or timeout reply (F.7). If the provider declines or does not answer, the *retry* flow returns control to 0.3.1 so that the next best candidate is tried. That loop is routed along an empty lane above the diagram, so it crosses nothing.

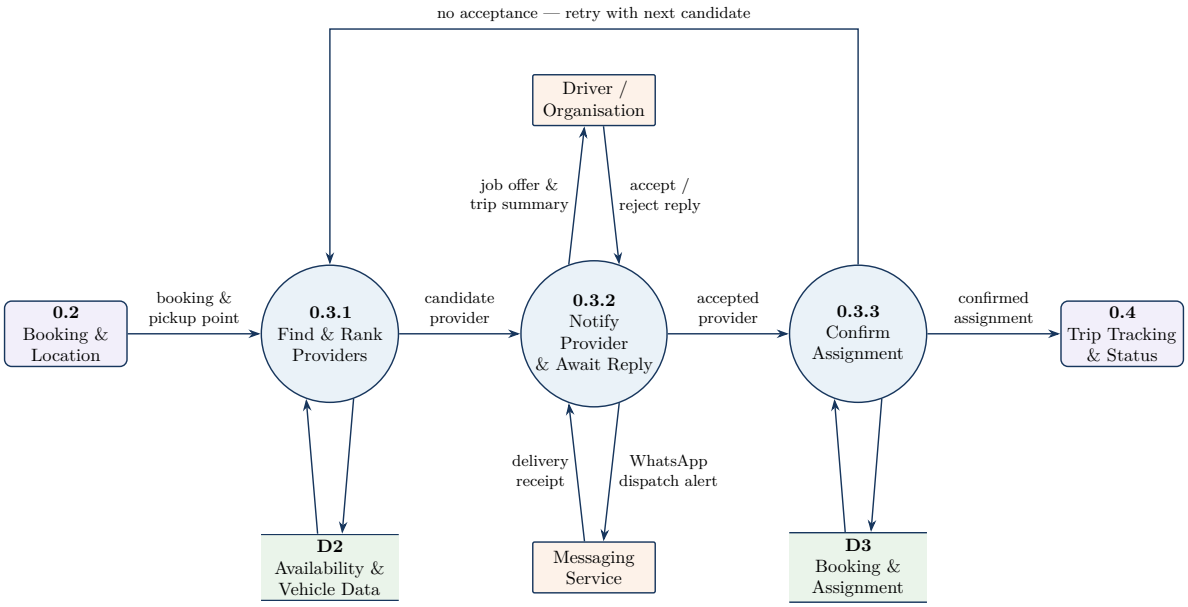


Figure 5: Level 2 DFD for Process 0.3: Matching and dispatch (F.5–F.7)

7 Level 2: Process 0.4 — Trip Tracking and Status

This diagram expands Process 0.4 and realises **F.8** (live tracking and ETA) and **F.9** (trip status updates). Position reports from the driver are stored in D4, the route and ETA are obtained from the Maps / GPS Service, and the resulting live view is published to the patient and pushed as alerts through the Messaging Service. On completion, the trip data is handed to Process 0.5.

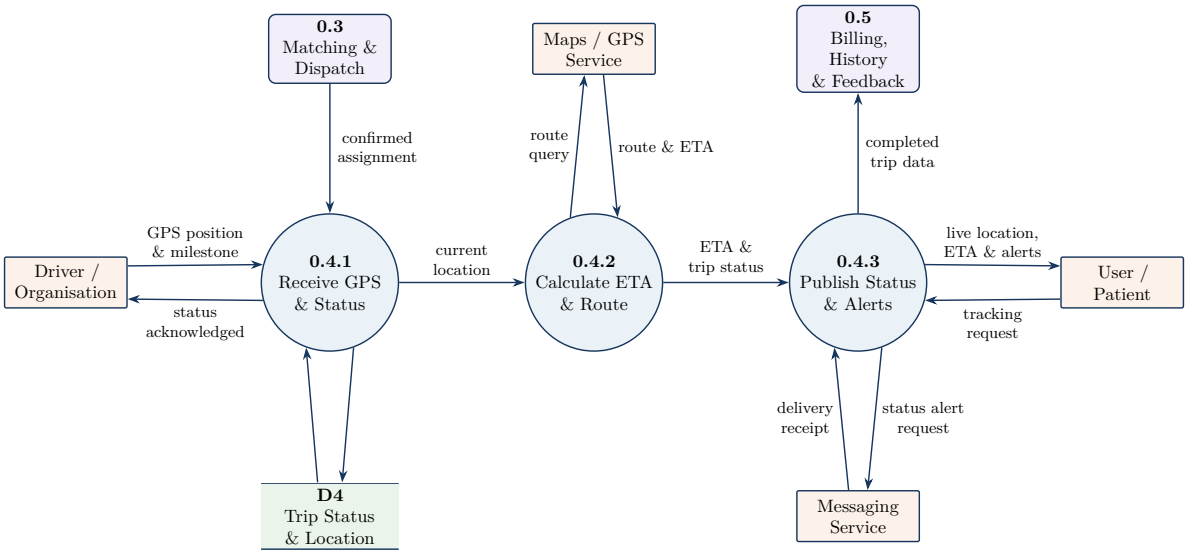


Figure 6: Level 2 DFD for Process 0.4: Trip tracking and status (F.8, F.9)

8 Level 2: Process 0.5 — Billing, History and Feedback

This diagram expands Process 0.5 and realises **F.10** (payment and billing), **F.11** (booking history and records) and **F.12** (rating and feedback). The fare is computed from the completed trip together with the booking details in D3, the charge is settled through the Payment Gateway, and the invoice, the payment result and the patient's rating are written to D5.

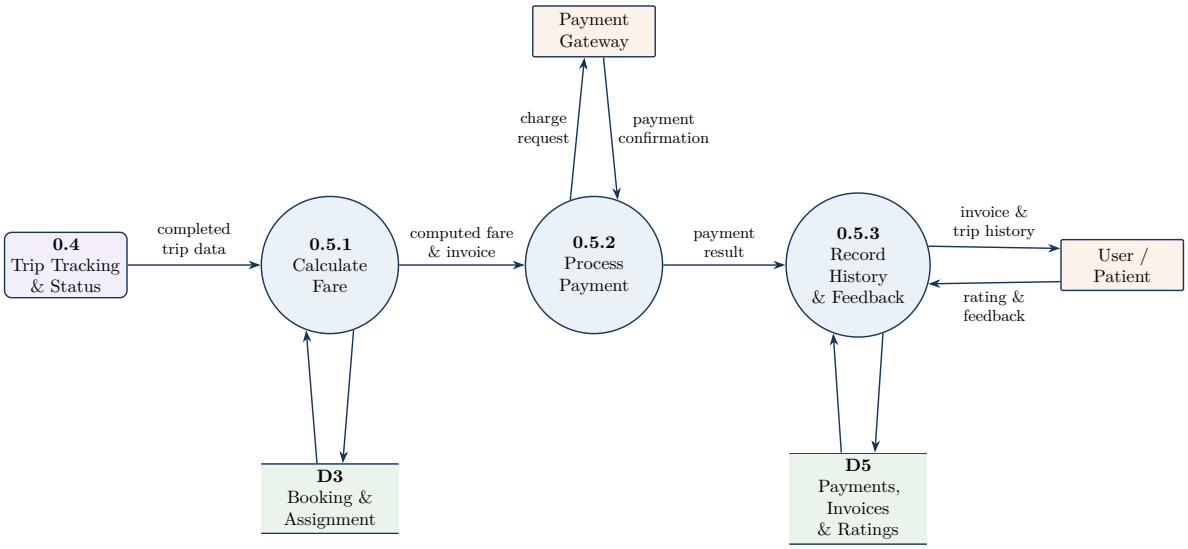


Figure 7: Level 2 DFD for Process 0.5: Billing, history and feedback (F.10–F.12)

9 Level 2: Process 0.6 — Administration and Analytics

This diagram expands Process 0.6 and realises **F.13** (administrative management) together with the reporting and alerting side of **F.14**. The administrator verifies and manages providers against D1, monitors live and past bookings from D3 and from Process 0.3, and generates operational and financial reports from D5, which are also pushed out as notifications through the Messaging Service. The administrator entity is drawn twice — as the source of the commands and as the recipient of the reports — so that no flow has to cross another.

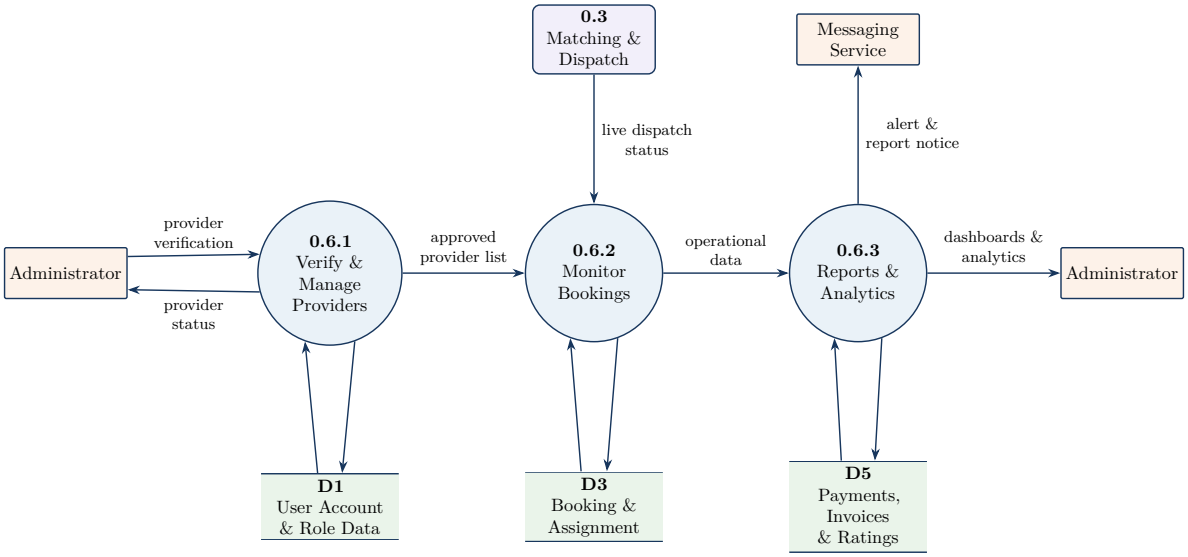


Figure 8: Level 2 DFD for Process 0.6: Administration and analytics (F.13, F.14)

DEPARTMENT OF INFORMATION TECHNOLOGY
NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA, SURATHKAL

Data Dictionary

for

Ambulance Booking System

(derived from the Software Requirements Specification,
the Data Flow Diagrams and the Structure Chart)

Prepared by:

Adithya Shyam D (241IT005)
Dev Patel (241IT023)
Ritesh Bhagat (241IT063)

Team 16

August 2026

Contents

1 Introduction **2**
1.1 Purpose . . . 2
1.2 Notation . . . 2
1.3 Conventions . . . 2
1.4 External Entities . . . 2

2 Data Element Definitions **4**

3 Composite Group Definitions **7**

4 Data Flow Definitions **7**
4.1 Level 0 — Context Diagram . . . 7
4.2 Level 1 — Top-Level DFD . . . 8
4.3 Level 2 — Process 0.1 Account and Access . . . 8
4.4 Level 2 — Process 0.2 Booking and Location . . . 9
4.5 Level 2 — Process 0.3 Matching and Dispatch . . . 9
4.6 Level 2 — Process 0.4 Trip Tracking and Status . . . 9
4.7 Level 2 — Process 0.5 Billing, History and Feedback . . . 10
4.8 Level 2 — Process 0.6 Administration and Analytics . . . 10

5 Data Store Definitions **11**

6 Structure Chart Data Couples **12**

7 Assumptions **13**

1 Introduction

1.1 Purpose

This document defines every data flow, data store, data couple and data element that appears in the design model of the Ambulance Booking System. It is the companion to the diagrams: they show *where* data moves, and this dictionary states *what* each named flow actually contains, down to atomic items with their types and validation rules. Names used here are identical to the labels on the diagrams, so an entry can be looked up directly from any figure.

It covers three models together. The Data Flow Diagrams (Levels 0–2) contribute the flows and the five data stores; the structure chart contributes the data couples passed between calling modules; and the Software Requirements Specification fixes the vocabulary and the permitted values, referenced throughout as SRS F.*n*.

The dictionary is organised bottom-up in the reading order that suits verification. Section 2 defines the atomic elements, Section 3 the reusable composite groups, Section 4 every data flow arranged by diagram level, Section 5 the five data stores D1–D5, and Section 6 the structure chart couples that are named differently from their DFD equivalents.

1.2 Notation

Definitions use DeMarco / Yourdon structured-analysis notation.

Symbol	Meaning
`=`	is composed of
`+`	and (sequence; both parts are present)
`( )`	optional; the enclosed part may be absent
`{ }`	iteration; the enclosed part repeats zero or more times
`[ | ]`	selection; exactly one of the alternatives separated by `|`
`**`	comment

1.3 Conventions

- Element names are lower case with underscores; they map one-to-one onto the field names intended for implementation.
- A flow named on two diagrams is defined once, at the level where it first appears, and is not repeated.
- Timestamps are ISO 8601 with an explicit timezone offset and are stored in UTC.
- All monetary amounts are Indian Rupees to two decimal places.
- Optional marking `( )` means the item may be absent from an occurrence of the structure; it does not mean the field is nullable in every state.

1.4 External Entities

The six entities of the context diagram, reproduced here so that the “where used” column of the flow tables can be read without the diagrams to hand.

Entity	Role
User / Patient	Places bookings, tracks the ambulance, pays and rates the trip.
Driver / Organisation	Receives dispatch offers, accepts or rejects them, and reports position and milestones.
Administrator	Verifies providers, monitors bookings and requests analytics.
Maps / GPS Service	Third-party geocoding and routing provider.
Messaging Service	WhatsApp Business API, push and SMS delivery for OTP and alerts.
Payment Gateway	Third-party settlement of fares.

2 Data Element Definitions

The atomic items of the model. Nothing in this table decomposes further; every composition in Sections 3–5 resolves into these.

Element	Type	Format / validation	Description
user_id	char(9)	'U' + 8 digits, e.g. U00010023	Unique identifier of any registered account holder (patient, driver, organisation or administrator).
full_name	varchar(60)	3-60 chars, letters and spaces	Legal name supplied at registration.
phone_no	char(10)	^[6-9][0-9]{9}$	Primary mobile number; doubles as the login identifier and the WhatsApp destination.
email_id	varchar(254)	RFC 5322 address	Optional contact address used for invoices and non-urgent notices.
password_hash	char(60)	bcrypt digest, cost 12	Stored credential. The plaintext password is never held or transmitted beyond TLS.
otp_code	char(6)	6 numeric digits	One-time password issued for phone verification and passwordless sign-in.
otp_expiry	timestamp	ISO 8601, issue time + 5 min	Instant after which an otp_code is rejected.
role_code	varchar(6)	[USER | DRIVER | ORG | ADMIN]	Determines the access scope granted by RBAC (SRS F.2).
verification_status	varchar(9)	[PENDING | VERIFIED | REJECTED | SUSPENDED]	Account or provider standing as set by the administrator (SRS F.13).
session_token	varchar(512)	JWT, HS256, 24 h lifetime	Bearer token proving an authenticated session.
token_expiry	timestamp	ISO 8601	Instant after which session_token must be refreshed.
registration_date	date	YYYY-MM-DD	Date the account was created.
provider_id	char(9)	'P' + 8 digits, e.g. P00004417	Unique identifier of a driver or ambulance organisation.
org_id	char(7)	'O' + 6 digits	Parent organisation of a driver, where the driver belongs to a fleet.
vehicle_reg_no	char(10)	^[A-Z]{2}[0-9]{2}[A-Z]{2}[0-9]{4}$	Registration plate of the ambulance, e.g. KA19AB1234.
ambulance_type	varchar(3)	[BLS | ALS | ICU | PTV]	Category of vehicle requested or offered (SRS F.3).
availability_status	varchar(9)	[AVAILABLE | BUSY | OFFLINE]	Live duty state of a provider, refreshed from the driver app.
licence_no	varchar(16)	State transport licence format	Driving licence presented for verification.
document_ref	varchar(200)	HTTPS object URL	Pointer to a stored licence, permit or registration scan.
last_seen_timestamp	timestamp	ISO 8601 with timezone	Instant of the most recent availability or location ping.
latitude	decimal(9,6)	-90.000000 to +90.000000	Geographic latitude in decimal degrees (WGS 84).
longitude	decimal(9,6)	-180.000000 to +180.000000	Geographic longitude in decimal degrees (WGS 84).
address_text	varchar(200)	free text	Human-readable street address of a pickup or drop point.

Element	Type	Format / validation	Description
landmark	varchar(100)	free text	Optional nearby reference supplied to help the driver locate the caller.
pincode	char(6)	6 digits	Indian postal code of the pickup area.
place_id	varchar(100)	provider-issued opaque key	Identifier returned by the mapping service for a resolved place.
booking_id	char(10)	'BK' + 8 digits, e.g. BK00031882	Unique identifier of a booking request (SRS F.3).
booking_timestamp	timestamp	ISO 8601 with timezone	Instant the request was placed.
booking_status	varchar(10)	[REQUESTED | MATCHING | ASSIGNED | ENROUTE | ARRIVED | ONBOARD | COMPLETED | CANCELLED | FAILED]	Lifecycle state of a booking.
notes	varchar(250)	free text	Optional free-text detail about the patient or emergency.
assignment_id	char(10)	'AS' + 8 digits	Unique identifier of one offer of one booking to one provider.
assignment_status	varchar(8)	[OFFERED | ACCEPTED | REJECTED | TIMEOUT]	Outcome of a dispatch offer (SRS F.7).
offer_expiry_timestamp	timestamp	ISO 8601, offer time + 45 s	Deadline after which silence is treated as TIMEOUT and the next candidate is tried.
retry_count	integer	0 to 5	Number of candidates already tried for this booking; caps the matching loop.
rank_score	decimal(5,2)	0.00 to 100.00	Composite suitability score combining distance, type match and rating.
distance_km	decimal(6,2)	0.00 to 9999.99	Road distance from the provider to the pickup point.
response_timestamp	timestamp	ISO 8601 with timezone	Instant the provider accepted or rejected an offer.
trip_id	char(10)	'TR' + 8 digits	Unique identifier of an executed trip, created once an assignment is confirmed.
trip_milestone	varchar(9)	[ASSIGNED | ENROUTE | ARRIVED | ONBOARD | COMPLETED]	Driver-declared stage of the trip (SRS F.9).
gps_timestamp	timestamp	ISO 8601, second precision	Instant a position report was sampled on the driver device.
speed_kmph	decimal(5,2)	0.00 to 250.00	Optional instantaneous speed reported with a position.
eta_minutes	integer	0 to 240	Estimated minutes remaining until arrival at the pickup point (SRS F.8).
route_polyline	text	encoded polyline, max 4000 chars	Geometry of the route returned by the mapping service.
route_distance_km	decimal(6,2)	0.00 to 9999.99	Total road distance of the computed route.
trip_start_time	timestamp	ISO 8601 with timezone	Instant the assignment was confirmed and the trip opened.
trip_end_time	timestamp	ISO 8601 with timezone	Instant the driver marked the trip COMPLETED.

Element	Type	Format / validation	Description
invoice_id	char(11)	'INV' + 8 digits	Unique identifier of an invoice raised on trip completion.
base_fare	decimal(8,2)	INR, >= 0.00	Flat component of the fare, fixed per ambulance_type.
distance_charge	decimal(8,2)	INR, >= 0.00	Distance component, route_distance_km multiplied by the per-km rate.
type_surcharge	decimal(8,2)	INR, >= 0.00	Optional premium applied to ALS and ICU vehicles.
tax_amount	decimal(8,2)	INR, >= 0.00	Statutory tax applied to the taxable components.
fare_amount	decimal(8,2)	INR, >= 0.00	Total payable, the sum of the components above (SRS F.10).
payment_mode	varchar(10)	[UPI | CARD | NETBANKING | CASH]	Settlement channel chosen by the patient.
payment_status	varchar(8)	[PENDING | SUCCESS | FAILED | REFUNDED]	Outcome reported by the payment gateway.
transaction_ref	varchar(40)	gateway-issued reference	Gateway handle used to reconcile or refund a charge.
payment_timestamp	timestamp	ISO 8601 with timezone	Instant the gateway confirmed the outcome.
rating_value	integer	1 to 5	Star rating given by the patient after completion (SRS F.12).
feedback_text	varchar(500)	free text	Optional written comment accompanying a rating.
message_id	varchar(40)	service-issued identifier	Handle of one outbound notification, used to correlate receipts.
message_channel	varchar(8)	[WHATSAPP | PUSH | SMS]	Transport used to deliver a notification (SRS F.6, F.14).
message_template_id	varchar(40)	approved template name	Pre-registered WhatsApp Business template invoked for the message.
delivery_state	varchar(9)	[SENT | DELIVERED | READ | FAILED]	Delivery state reported back by the messaging service.
delivery_timestamp	timestamp	ISO 8601 with timezone	Instant the delivery state was reported.
report_id	char(9)	'RPT' + 6 digits	Unique identifier of a generated report.
report_type	varchar(16)	[RESPONSE_TIME | DEMAND | REVENUE | PROVIDER_PERF]	Class of analytics requested by the administrator (SRS F.13).
from_date	date	YYYY-MM-DD	Inclusive start of a reporting window.
to_date	date	YYYY-MM-DD	Inclusive end of a reporting window; must not precede from_date.
metric_value	decimal(10,2)	signed	One computed figure within a report.
query_filter	varchar(200)	field=value pairs	Restriction applied by the administrator when browsing bookings or providers.
response_time_sec	integer	0 to 86400	Seconds elapsed between booking_timestamp and the confirmed assignment.
remarks	varchar(200)	free text	Optional administrator note recorded against a verification decision.

3 Composite Group Definitions

Structures that recur across several flows. Defining them once keeps the flow definitions short and guarantees that, for example, a pickup point has the same shape everywhere it appears.

Group	Composition	Notes
location	= latitude + longitude	Bare coordinate pair, used wherever a point is needed without an address.
pickup_point	= location + address_text + (landmark) + (pincode)	Fully resolved origin of a booking, as written to D3.
credentials	= phone_no + [password_hash | otp_code]	Either factor is sufficient; the system supports passwordless sign-in.
provider_ref	= provider_id + (org_id) + vehicle_reg_no + ambulance_type	Compact identification of a provider and the vehicle offered.
fare_breakdown	= base_fare + distance_charge + (type_surcharge) + tax_amount + fare_amount	Itemised charge shown on the invoice.
date_range	= from_date + to_date	Reporting window supplied with an analytics request.

4 Data Flow Definitions

Every labelled arrow in the DFD model, grouped by the diagram on which it appears. The final column gives the source and destination as drawn.

4.1 Level 0 — Context Diagram

Flow	Composition	Where used
booking_request	= user_id + ambulance_type + pickup_point + booking_timestamp + (notes)	User / Patient → 0
status_and_confirmation	= booking_id + booking_status + (provider_ref) + (eta_minutes)	0 → User / Patient
dispatch_order	= booking_id + provider_id + pickup_point + ambulance_type + offer_expiry_timestamp	0 → Driver / Organisation
status_update	= booking_id + trip_id + trip_milestone + location + gps_timestamp	Driver / Organisation → 0
management_commands	= user_id + role_code + [provider_verification | query_filter | report_type + date_range]	Administrator → 0
reports_and_dashboards	= report_id + report_type + date_range + { metric_value }	0 → Administrator
location_query	= [address_lookup | route_query]	0 → Maps / GPS Service
eta_and_route	= route_polyline + route_distance_km + eta_minutes	Maps / GPS Service → 0
otp_and_alert_request	= message_channel + message_template_id + phone_no + [otp_code | booking_id + pickup_point]	0 → Messaging Service

Flow	Composition	Where used
delivery_status	= message_id + delivery_state + delivery_timestamp	Messaging Service → 0
payment_request	= invoice_id + booking_id + fare_amount + payment_mode	0 → Payment Gateway
payment_confirmation	= transaction_ref + payment_status + payment_timestamp + fare_amount	Payment Gateway → 0

4.2 Level 1 — Top-Level DFD

Flow	Composition	Where used
credentials_and_sign_up	= full_name + phone_no + (email_id) + credentials + role_code	User / Patient → 0.1
session_and_auth_result	= user_id + session_token + token_expiry + role_code	0.1 → User / Patient
authenticated_session	= user_id + role_code + session_token	0.1 → 0.2
accept_reject	= assignment_id + provider_id + assignment_status + response_timestamp	Driver / Organisation → 0.3
confirmed_assignment	= booking_id + assignment_id + provider_ref + pickup_point + assignment_status + trip_start_time	0.3 → 0.4
completed_trip_data	= trip_id + booking_id + provider_id + trip_start_time + trip_end_time + route_distance_km + trip_milestone	0.4 → 0.5
billing_and_history	= invoice_id + fare_breakdown + payment_status + { booking_id + booking_timestamp + booking_status }	0.5 → User / Patient
administrative_commands	= user_id + role_code + [provider_verification | query_filter | report_type + date_range]	Administrator → 0.6
account_and_provider_actions	= provider_id + verification_status + (remarks)	0.6 → 0.1

4.3 Level 2 — Process 0.1 Account and Access

Flow	Composition	Where used
sign_up_and_login_data	= full_name + phone_no + (email_id) + credentials + role_code	Actor → 0.1.1, 0.1.2
otp_send_request	= phone_no + otp_code + message_channel + message_template_id	0.1.1 → Messaging Service
otp_code_and_delivery_status	= message_id + delivery_state + delivery_timestamp	Messaging Service → 0.1.1
otp_prompt_and_sign_up_status	= phone_no + otp_expiry + verification_status	0.1.1 → Actor
verified_identity	= user_id + phone_no + verification_status + registration_date	0.1.1 → 0.1.2

Flow	Composition	Where used
authenticated_user	= user_id + role_code + verification_status	0.1.2 → 0.1.3
session_token_and_dashboard	= session_token + token_expiry + role_code + user_id	0.1.3 → Actor

4.4 Level 2 — Process 0.2 Booking and Location

Flow	Composition	Where used
ambulance_type_and_pickup_point	= user_id + ambulance_type + raw_pickup_input	User / Patient → 0.2.1
raw_pickup_input	= [address_text | location] + (landmark)	0.2.1 → 0.2.2
address_lookup	= address_text + (pincode)	0.2.2 → Maps / GPS Service
geocoded_coordinates	= location + address_text + (place_id)	Maps / GPS Service → 0.2.2
validated_location	= pickup_point + (place_id)	0.2.2 → 0.2.3
booking_ref_and_confirmation	= booking_id + booking_status + booking_timestamp	0.2.3 → User / Patient

4.5 Level 2 — Process 0.3 Matching and Dispatch

Flow	Composition	Where used
booking_and_pickup_point	= booking_id + ambulance_type + pickup_point + booking_timestamp	0.2 → 0.3.1
candidate_provider	= provider_ref + distance_km + rank_score + availability_status	0.3.1 → 0.3.2
job_offer_and_trip_summary	= assignment_id + booking_id + pickup_point + ambulance_type + offer_expiry_timestamp	0.3.2 → Driver / Organisation
whatsapp_dispatch_alert	= message_template_id + message_channel + phone_no + booking_id + pickup_point	0.3.2 → Messaging Service
delivery_receipt	= message_id + delivery_state + delivery_timestamp	Messaging Service → 0.3.2, 0.4.3
accept_reject_reply	= assignment_id + provider_id + assignment_status + response_timestamp	Driver / Organisation → 0.3.2
retry_with_next_candidate	= booking_id + retry_count + { provider_id }	0.3.2 → 0.3.1 (loop)
accepted_provider	= provider_ref + assignment_id + assignment_status + response_timestamp	0.3.2 → 0.3.3

4.6 Level 2 — Process 0.4 Trip Tracking and Status

Flow	Composition	Where used
gps_position_and_milestone	= trip_id + provider_id + location + gps_timestamp + trip_milestone + (speed_kmph)	Driver / Organisation → 0.4.1
status_acknowledged	= trip_id + trip_milestone + gps_timestamp	0.4.1 → Driver / Organisation
current_location	= trip_id + location + gps_timestamp	0.4.1 → 0.4.2
route_query	= location + pickup_point	0.4.2 → Maps / GPS Service
route_and_eta	= route_polyline + route_distance_km + eta_minutes	Maps / GPS Service → 0.4.2
eta_and_trip_status	= booking_id + trip_id + eta_minutes + trip_milestone	0.4.2 → 0.4.3
tracking_request	= user_id + booking_id	User / Patient → 0.4.3
live_location_eta_and_alerts	= trip_id + location + eta_minutes + trip_milestone	0.4.3 → User / Patient
status_alert_request	= message_channel + message_template_id + phone_no + trip_milestone	0.4.3 → Messaging Service

4.7 Level 2 — Process 0.5 Billing, History and Feedback

Flow	Composition	Where used
computed_fare_and_invoice	= invoice_id + booking_id + fare_breakdown	0.5.1 → 0.5.2
charge_request	= invoice_id + booking_id + fare_amount + payment_mode + user_id	0.5.2 → Payment Gateway
payment_result	= invoice_id + payment_status + transaction_ref + payment_timestamp	0.5.2 → 0.5.3
rating_and_feedback	= booking_id + user_id + rating_value + (feedback_text)	User / Patient → 0.5.3
invoice_and_trip_history	= { invoice_id + booking_id + fare_amount + payment_status + booking_timestamp }	0.5.3 → User / Patient

4.8 Level 2 — Process 0.6 Administration and Analytics

Flow	Composition	Where used
provider_verification	= provider_id + licence_no + vehicle_reg_no + { document_ref } + (remarks)	Administrator → 0.6.1
provider_status	= provider_id + verification_status + (remarks)	0.6.1 → Administrator
approved_provider_list	= { provider_ref + verification_status }	0.6.1 → 0.6.2
live_dispatch_status	= { booking_id + booking_status + provider_id + eta_minutes }	0.3 → 0.6.2
operational_data	= date_range + { booking_id + booking_status + response_time_sec }	0.6.2 → 0.6.3

Flow	Composition	Where used
alert_and_report_notice	= message_channel + message_template_id + phone_no + report_id	0.6.3 → Messaging Service
dashboards_and_analytics	= report_id + report_type + date_range + { metric_value }	0.6.3 → Administrator

5 Data Store Definitions

The five stores of the model, numbered as in the DFD document. Each is an iteration, since a store holds many occurrences of its record.

D1 — User Account and Role Data

Composition

```
{ user_id + full_name + phone_no + ( email_id ) + password_hash + role_code + verification_status + registration_date + ( licence_no ) + ( org_id ) + ( remarks ) }
```

Notes Written by 0.1.1 and 0.6.1; read by 0.1.2 and 0.6.1. One occurrence per account, keyed on user_id.

D2 — Availability and Vehicle Data

Composition

```
{ provider_id + ( org_id ) + vehicle_reg_no + ambulance_type + availability_status + location + last_seen_timestamp }
```

Notes Written by the driver app through 0.4.1; read by 0.3.1 when ranking candidates. Keyed on provider_id.

D3 — Booking and Assignment

Composition

```
{ booking_id + user_id + ambulance_type + pickup_point + booking_timestamp + booking_status + ( notes ) + ( assignment_id + provider_id + assignment_status + offer_expiry_timestamp + retry_count + response_timestamp ) }
```

Notes Written by 0.2.3 and 0.3.3; read by 0.3.1, 0.5.1 and 0.6.2. Keyed on booking_id; the assignment group is absent until a provider accepts.

D4 — Trip Status and Location

Composition

```
{ trip_id + booking_id + provider_id + { location + gps_timestamp + ( speed_kmph ) } + trip_milestone + eta_minutes + route_polyline + route_distance_km + trip_start_time + ( trip_end_time ) }
```

Notes Written by 0.4.1 and 0.4.2; read by 0.4.3 and 0.5.1. The inner iteration is the GPS trace, one entry per position report.

D5 — Payments, Invoices and Ratings

Composition

```
{ invoice_id + booking_id + fare_breakdown + payment_mode + payment_status + transaction_ref
+ payment_timestamp + ( rating_value ) + ( feedback_text ) }
```

Notes Written by 0.5.2 and 0.5.3; read by 0.5.3 and 0.6.3. Keyed on invoice_id; rating fields stay absent until the patient responds.

6 Structure Chart Data Couples

The structure chart decomposes the same system into calling modules and annotates each call with the data couples passed in and returned. Those couples carry the same information as the DFD flows, but the chart labels several of them more tersely because the annotation has to fit beside an arrow.

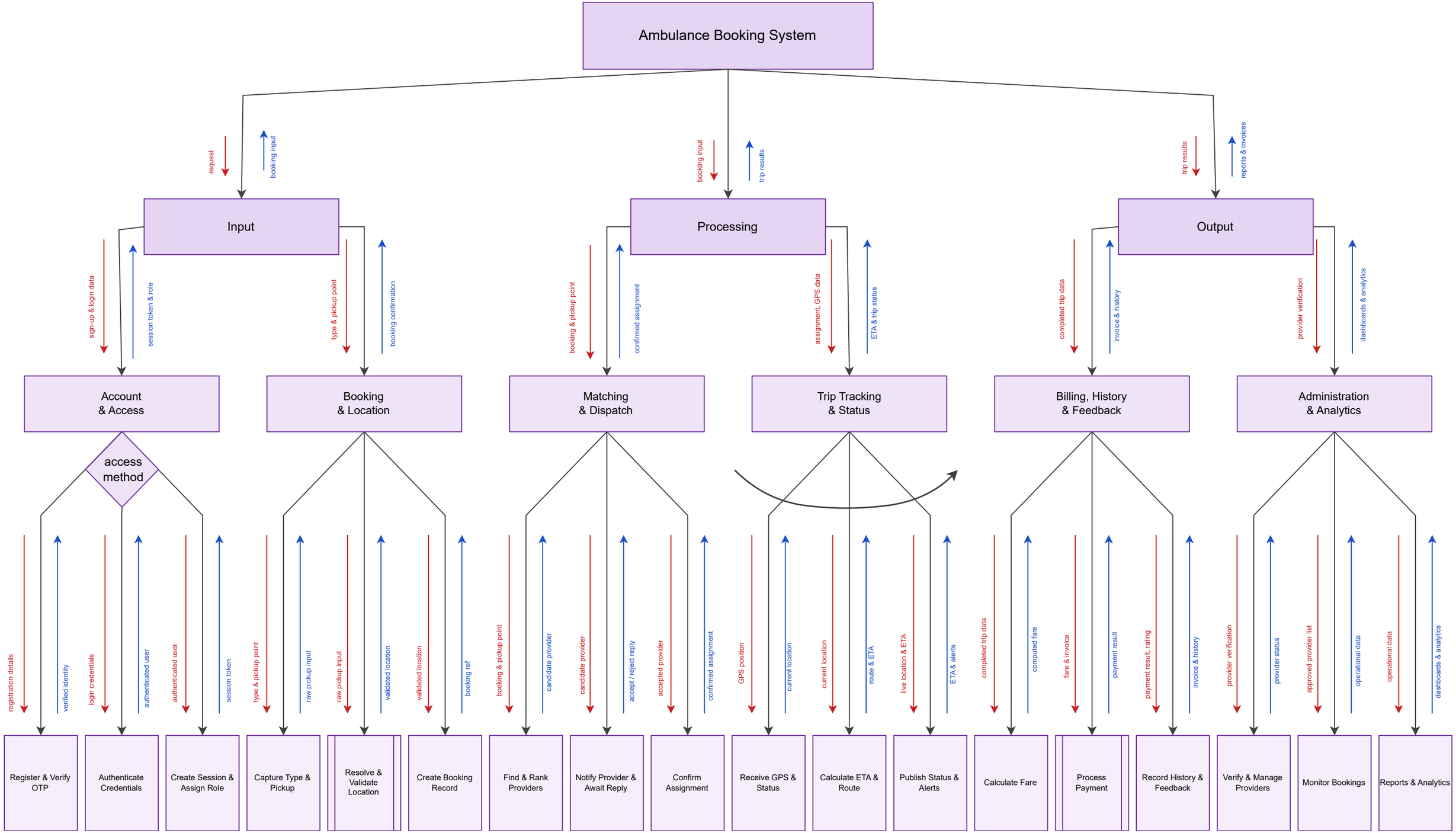
Couples whose label already matches a flow in Section 4 are not repeated. The table below defines the remaining fourteen: the terse variants, and the couples that exist only on the chart. Where a couple is simply a shorter name for a flow already defined, the equivalence is recorded as a comment so the two models can be checked against each other.

Couple	Composition	Passed between
session_token_and_role	= session_token + token_expiry + role_code + user_id	0.1 → 0. ** same content as session_token_and_dashboard **
type_and_pickup_point	= ambulance_type + raw_pickup_input	0. → 0.2; 0.2 → 0.2.1
booking_confirmation	= booking_id + booking_status + booking_timestamp	0.2 → 0. ** same content as booking_ref_and_confirmation **
assignment_gps_data	= confirmed_assignment + gps_position_and_milestone	0. → 0.4
invoice_and_history	= { invoice_id + booking_id + fare_amount + payment_status + booking_timestamp }	0.5 → 0.; 0.5.3 → 0.5
registration_details	= full_name + phone_no + (email_id) + role_code	0.1 → 0.1.1
login_credentials	= credentials + role_code	0.1 → 0.1.2
booking_ref	= booking_id	0.2.3 → 0.2
gps_position	= trip_id + provider_id + location + gps_timestamp	0.4 → 0.4.1
live_location_and_eta	= trip_id + location + eta_minutes	0.4 → 0.4.3
eta_and_alerts	= booking_id + eta_minutes + trip_milestone + message_channel	0.4.3 → 0.4
computed_fare	= invoice_id + fare_breakdown	0.5.1 → 0.5
fare_and_invoice	= invoice_id + booking_id + fare_breakdown + payment_mode	0.5 → 0.5.2
payment_result_and_rating	= payment_result + rating_value + (feedback_text)	0.5 → 0.5.3

7 Assumptions

The SRS and the DFDs name every flow and store but do not fix field-level detail. The following were chosen here and should be confirmed before implementation.

- Identifier formats (the `U`, `P`, `BK`, `AS`, `TR`, `INV` and `RPT` prefixes) are a convention adopted in this dictionary; the SRS does not prescribe them.
- `phone_no` is validated as a ten-digit Indian mobile number, consistent with the SRS assumption of domestic operation.
- The dispatch offer window is taken as 45 seconds and the matching loop is capped at five candidates, giving effect to the automatic reassignment required by SRS F.7 and NF.1 without the SRS stating a figure.
- OTP validity is taken as five minutes.
- Fares are modelled as base plus distance plus an optional type surcharge plus tax, which matches the SRS F.10 statement that fare depends on distance and ambulance type.
- `ambulance_type` is restricted to the four categories named in SRS F.3.


Ambulance Booking System
request
booking input
booking input
trip results
trip results
reports & invoices
Input
Processing
Output
sign-up & login data
session token & role
type & pickup point
booking confirmation
booking & pickup point
confirmed assignment
assignment, GPS data
ETA & trip status
completed trip data
invoice & history
provider verification
dashboards & analytics
Account & Access
Booking & Location
Matching & Dispatch
Trip Tracking & Status
Billing, History & Feedback
Administration & Analytics
access method
registration details
verified identity
login credentials
authenticated user
authenticated user
session token
type & pickup point
raw pickup input
raw pickup input
validated location
validated location
booking ref
booking & pickup point
candidate provider
candidate provider
accept / reject reply
accepted provider
confirmed assignment
GPS position
current location
current location
route & ETA
live location & ETA
ETA & alerts
completed trip data
computed fare
fare & invoice
payment result
payment result, rating
invoice & history
provider verification
provider status
approved provider list
operational data
operational data
dashboards & analytics
Register & Verify OTP
Authenticate Credentials
Create Session & Assign Role
Capture Type & Pickup
Resolve & Validate Location
Create Booking Record
Find & Rank Providers
Notify Provider & Await Reply
Confirm Assignment
Receive GPS & Status
Calculate ETA & Route
Publish Status & Alerts
Calculate Fare
Process Payment
Record History & Feedback
Verify & Manage Providers
Monitor Bookings
Reports & Analytics